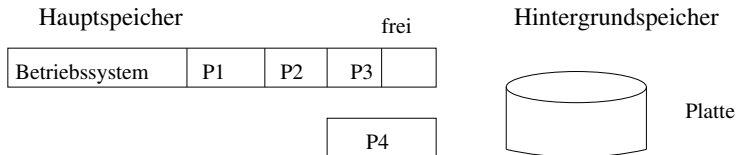


4. Speicherverwaltung

*"Ideally one would desire an indefinitely large memory capacity such that any particular ... word would be immediately available ... We are ... forced to recognize the possibility of constructing a **hierarchy of memories**, each of which has greater capacity than the preceding but which is less quickly accessible."*

A. W. Burks, H. H. Goldstine und J. von Neumann, 1946 (gefunden im Patterson/Hennessy)



Wiederholung: Speicherhierarchie

- ① Register
- ② Cache
- ③ Hauptspeicher/Primärspeicher:
Flüchtiger Speicher (volatile memory): Wenn die Stromzufuhr unterbrochen ist, gehen alle Daten verloren.
⇒ geeignet für Programme in Ausführung

Random-Access Memory (RAM): is a form of computer memory that can be read and changed in any order, typically used to store working data and machine code. A random-access memory device allows data items to be read or written in almost the same amount of time irrespective of the physical location of data inside the memory.

Technologien:

- ① SRAM (Static RAM):
Behält seine Dateninformation, solange die Betriebsspannung anliegt.
- ② DRAM (Dynamic RAM) (seit 1975): Im Gegensatz zu SRAM benötigt der DRAM ein periodisches (dynamisches) Auffrischen (Re-Writing) zur Vermeidung von Datenverlust in jeder Datenzelle
Zugriffszeit DRAM: ca. 80 Nanosekunden $\Rightarrow$ 100 000-mal schneller als Festplatte

- ④ Sekundärspeicher (Hintergrundspeicher):
nichtflüchtiger (non-volatile) Speicher, der dauerhaftes
Speichern von Daten ermöglicht

Beispiel:

- magnetische Festplatten (seit 1965):
 - + Kosten pro Megabyte geringer als DRAM
 - Zugriffszeiten aufgrund der mechanischen Komponenten
deutlich länger als bei DRAM: Festplatte 5-15 ms
- eingebettete Systeme: FLASH-Speicher: nichtflüchtiger
Halbleiterspeicher
Speicherkarten für Digitalkameras, Mobiltelefone
auch als Solid-State-Drives (SSD) als Alternative zu den
bisherigen magnetischen Festplatten

Definition 25:

Unter **Swapping** versteht man das Verlagern von Prozessen zwischen Hauptspeicher und Hintergrundspeicher.

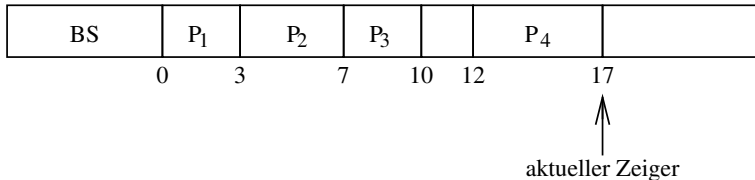
Idee:

- **Auslagern (swap out)** blockierter Prozesse auf den Hintergrundspeicher.
- **Einlagern (swap in)** nach Übergang in den Zustand „bereit“.

Probleme:

- Aufwand für Ein- und Auslagern,
- Belegungsstrategie beim Einlagern.

Beispiel:



Ablauf:

Prozeß P_2 terminiert

Prozeß P_5 der Größe 1 einlagern

verkettete Liste mit Einträgen:

Belegungsstrategien

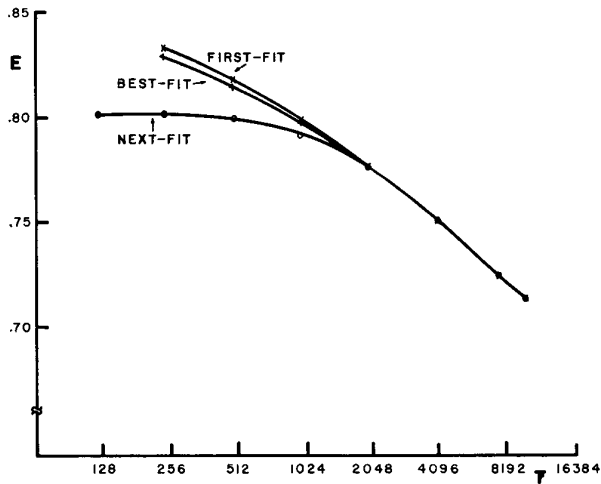
- First Fit:** Der Prozeß wird in die erste hinreichend große Lücke eingelagert (Liste von vorn durchlaufen).
- Next Fit:** Analog zu First Fit von der aktuellen Zeigerposition aus.
- Best Fit:** Der Prozeß wird in diejenige Lücke eingelagert, die den kleinsten Rest ergibt.
- Worst Fit:** Der Prozeß wird in die größte Lücke eingelagert.

Ziel: Externe Fragmentierung (Externer Verschnitt), d.h. ungenutzten Hauptspeicherplatz, minimieren!

Literatur:

John E. Shore: *On the External Storage Fragmentation Produced by First-Fit and Best-Fit Allocation Strategies*, Communications of the ACM, Vol. 18, Number 8, 1975.

Fig. 1.



entnommen: Carter Bays: *A comparison of next-fit, first-fit, and best-fit*,
 Communications of the ACM, Volume 20 Issue 3, March 1977, pages 191 - 192.

Buddy-Verfahren (Knuth '73):

n Listen für Lücken der Länge $1, 2, 4, 8, \dots, 2^{n-1}$

Belegen:

- ① Runde die Größe der Anforderung auf die nächste Zweierpotenz 2^k auf
- ② Suche die erste nicht-leere 2^m -Liste mit $m \geq k$
- ③ Falls $m = k$: Wähle den ersten Block, andernfalls unterteile den ersten Block aus der 2^m -Liste!

Freigeben (Merge *Best Buddies*):

- ① Falls sich der freigegebene Block auf seiner 2^k -Liste verschmelzen läßt, tue dies! Ggf. auf der 2^{k+1} -Liste wiederholen, usw.

⇒ reduziert externen Verschnitt!

aber: Es entsteht **interner Verschnitt** (interne Fragmentierung), da Prozeßgrößen i.a. keine 2er-Potenz sind.

Nachteil vom Swapping:

① Fragmentierung

Abhilfe:

- Buddy-Verfahren $\Rightarrow$ interne Fragmentierung
- **Kompaktifizierung:** Verlagern von Prozessen im Hauptspeicher, um den zusammenhängenden freien Adreßraum zu vergrößern.
 $\Rightarrow$ hoher Kopieraufwand!

② Die Prozeßgröße wird durch die Hauptspeichergröße beschränkt!

Lokalitätsprinzip:= Prozesse greifen während eines vorgegebenen Zeitintervalls nur auf eine Teilmenge ihres Adreßraums zu. $\implies$ Diese Teilmenge ist durch eine höhere Zugriffswahrscheinlichkeit gekennzeichnet.

Ursachen:

- sequentielle Instruktionsfolgen
- iterative Anweisungen (Schleifen)
- Prozeduraufrufe
- modulares Programmieren

Fazit: Prozesse können laufen, ohne daß sich der komplette Adreßraum im Hauptspeicher befindet $\implies$ Gesamteinlagerung von Prozessen ist überflüssig!

Idee: Programme benutzen **virtuelle** Adressen!

- Der **virtuelle** und der **physikalische Adreßraum** werden in Blöcke gleicher Größe aufgeteilt.
Üblich: Blöcke von 512 bytes – 8 KByte
Ein Block im virtuellen Adreßraum heißt **Seite (page)**, ein Block im physikalischen Adreßraum heißt **Kachel** bzw. **Seitenrahmen (page frame)**.
! Häufig werden auch physikalische Blöcke “pages” genannt !
- Zur Laufzeit muß die virtuelle Adresse dann auf die physikalische Adresse im Hauptspeicher abgebildet werden.
Die **Seitentabelle** realisiert die Zuordnung:

Seite $\mapsto$ Seitenrahmen

Beispiel: Der Prozeß P besteht aus 5 Blöcken $A_1, A_2, \dots, A_5$ der Größe 4K mit den Seitennummern 1, 2, ..., 5.

Hauptspeicherbelegung:

0	1	2		8	
	A_5	A_1		A_4	

Seitentabelle pro Prozeß:

Seitennr.	Kachelnr.
1	2
2	—
3	—
4	8
5	1

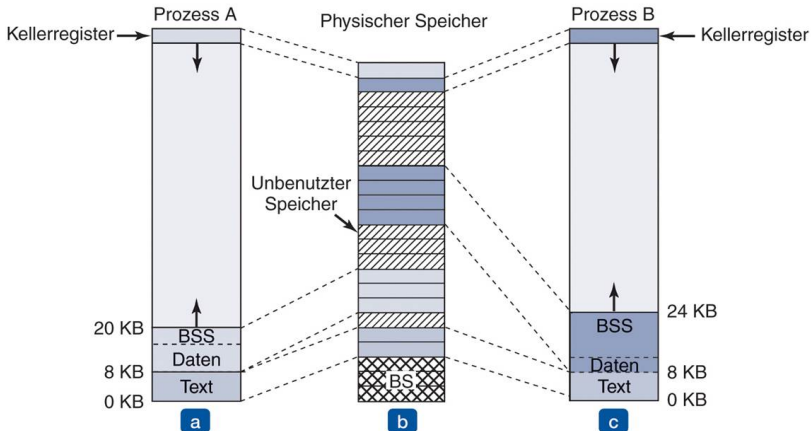


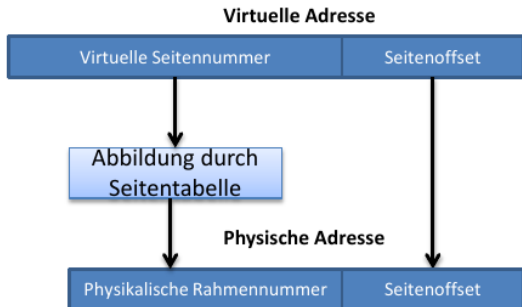
Abbildung 10.12: (a) Virtueller Adressraum von Prozess A (b) Physischer Speicher (c) Virtueller Adressraum von Prozess B

Quelle: Andrew Tanenbaum: Moderne Betriebssysteme

Beispiel für ein Speicherlayout, wenn zwei Prozesse ein gemeinsames Textsegment haben
(Zwei Benutzer starten zum Beispiel denselben Texteditor.)

Adreßumsetzung

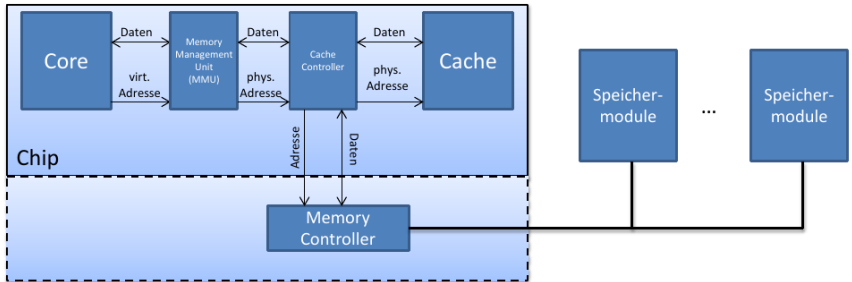
virtuelle Adresse: (Seitennummer, Offset innerhalb der Seite)



Quelle: VL-Skript Mario Schölzel

Realisierung einer Seitentabelle

Die **Memory Management Unit (MMU)** übersetzt die virtuellen Adressen in physikalische Adressen.



Quelle: VL-Skript Mario Schölzel

Ziel:

- ① schneller Zugriff,
- ② möglichst wenig Speicherplatzbedarf.

zu 1.: **Translation Lookaside Buffer (TLB)**: schneller assoziativer Zwischenspeicher in der MMU, der

- Anfangsadresse der Seitentabelle und
- ca. 32-1024 „aktuell benötigte“ Einträge der Seitentabelle enthält. Die Verwaltung von Treffern findet vollständig in Hardware statt. Trefferrate typischerweise $> 99\%$.

zu 2. Notwendiger Kompromiß:

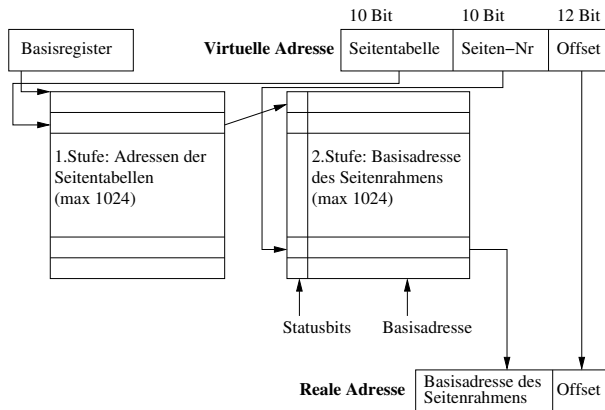
kleine Seitengröße $\Rightarrow$ Seitentabelle wächst!

große Seitengröße $\Rightarrow$ interner Verschnitt wächst!

Um Platz für die Seitentabellen im Hauptspeicher zu sparen
 $\Rightarrow$ Mehrstufige Seitentabelle

Mehrstufige Seitentabellen

Beispiel: 2-stufige Seitentabelle



Definition 26:

Ein **Seitenfehler** tritt auf, falls während der Programmausführung ein Zugriffswunsch auf eine Seite erfolgt, die sich **nicht** im Hauptspeicher befindet.

⇒ **Demand-Paging** := Seitenwechsel auf Anforderung

Gefahr: **Seitenflattern** := Das Betriebssystem beschäftigt sich nur noch mit dem Ein-/Auslagern von Seiten.

Demand-Paging-Verfahren

Definition 27:

Ein Programmablauf wird durch eine **Seitenzugriffsfolge**, auch **Referenzstring** genannt,

$$W = s[1]s[2] \dots s[T]$$

beschrieben, wobei $s[t] \in \mathbf{N}$ die Seitennummer der Seite, auf die zum Zeitpunkt t zugegriffen wird, bezeichnet.

Demand-Paging-Verfahren:

- $s[t]$ zum Zeitpunkt t nicht im Hauptspeicher $\Rightarrow$ einlagern!
- Hauptspeicher belegt $\Rightarrow$ Es muß eine Seite $\tilde{s}$ ausgewählt werden, die ausgelagert wird.
- Falls $\tilde{s}$ modifiziert ist $\Rightarrow \tilde{s}$ auf den Hintergrundspeicher zurückschreiben. Andernfalls kann $\tilde{s}$ überschrieben werden.

Ersetzungsstrategien (Page Replacement Algorithms)

Optimierungsziel: möglichst wenig Seitenfehler

FIFO: „älteste“ Seite wird ausgelagert.

- einfach zu implementieren mit einer einfach verketteten Liste

Beispiel mit $m=3$

$\hat{=}$ zufällige Auswahl

Least Recently Used (LRU): Die am längsten nicht benutzte Seite wird ausgelagert.

Aufwand: Bei jedem Seitenzugriff muß die Verwaltungsstruktur (z.B. doppelt-verkettete Liste) aktualisiert werden!

Belady's optimaler Algorithmus:

Die Seite mit dem größten **Vorwärtsabstand** wird ausgelagert.

Vorwärtsabstand der Seite x zum Zeitpunkt t eines Programms mit Laufzeit T :

$$d(x, t) := \begin{cases} k, & \text{falls } x \notin \{s[t+1], \dots, s[t+k-1]\} \text{ und } s[t+k] = x \\ \infty, & \text{falls } x \notin \{s[t+1], \dots, s[T]\} \end{cases}$$

- Optimaler Algorithmus,
- nur für die Beurteilung anderer Verfahren interessant, da $d(x, t)$ nicht a priori bekannt ist.

Definition 28:

$F(A, m, W)$ bezeichne die Anzahl Seitenfehler eines Paging-Algorithmus A zu gegebenem Referenzstring W und Hauptspeichergröße m .

Ein Paging-Algorithmus heißt **gutartig**, falls

$$\forall W \in IN^{IN} \quad \forall m \in IN : F(A, m+1, W) \leq F(A, m, W).$$

Definition 29:

$S(A, m, W, t)$ bezeichne die Menge der zum Zeitpunkt t gemäß dem Paging-Algorithmus A und dem Referenzstring W in den Hauptspeicher der Größe m eingelagerten Seiten.

Ein Algorithmus heißt **Stack-Algorithmus**, falls

$$\forall m \in IN \quad \forall t \in IN \quad \forall W \in IN^{IN} : S(A, m, W, t) \subseteq S(A, m+1, W, t).$$

Bem.: Stack-Algorithmen sind gutartig.

Bem.:

- ① FIFO ist nicht gutartig (Belady's Anomalie).

im Beispiel $m = 4$

W	0	1	2	3	0	1	4	0	1	2	3	4
jüngste Seite	0	1	2	3	3	3	4	0	1	2	3	4
	-	0	1	2	2	2	3	4	0	1	2	3
	-	-	0	1	1	1	2	3	4	0	1	2
älteste Seite	-	-	-	0	0	0	1	2	3	4	0	1
Seitenfehler	*	*	*	*			*	*	*	*	*	*

$\Rightarrow$ 10 Seitenfehler!

- ② LRU ist gutartig.

Working-Set-Modell (Denning 1970)

Definition 30:

Gegeben sei der Referenzstring $s[1] s[2] \dots s[t]$ eines Programms zum Zeitpunkt t .

Dann heißt

$$W(t, T) := \{i \in \mathbb{N} \mid i \in \{s[t-T], s[t-T+1], \dots, s[t-1]\}\}$$

Working-Set (Arbeitsmenge) des Programms zum Zeitpunkt t .

T := heißt **Working-Set-Parameter**.

Ein **Working-Set-Algorithmus** basiert auf folgender Eigenschaft:

Die Seiten des Working-Set werden im Hauptspeicher gehalten.

Jede beliebige Seite, die nicht zum Working-Set $W(t, T)$ gehört, kann ausgelagert werden.

Wahl von T : $0 < T < m$

Beispiel: Hauptspeicher mit $m=4$ Seitenrahmen

OBDA: Unter den Seiten, die nicht zum Working-Set gehören, wähle diejenige mit der kleinsten Seitennummer aus

$T = 1$

m	5	2	6	4	3	2	1	3	2	1	
1	5	5	5	5							
2		2	2	2							
3			6	6							
4				4							
SF	*	*	*	*							10 SF

$T = 2$

m	5	2	6	4	3	2	1	3	2	1	
1	5	5	5	5							
2		2	2	2							
3			6	6							
4				4							
SF	*	*	*	*							7 SF

$$T = 3$$

m	5	2	6	4	3	2	1	3	2	1	
1	5	5	5	5							
2		2	2	2							
3			6	6							
4				4							
SF	*	*	*	*							6 SF

Statische/Dynamische Seitenersetzungsverfahren

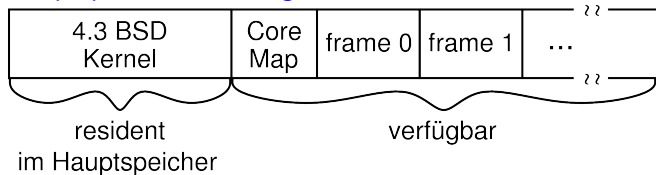
Bei **statischen (lokalen) Verfahren** darf jeder Prozeß gleich viel Seitenrahmen bis zu einer vorgegebenen Maximalzahl im Hauptspeicher belegen.

Im Gegensatz dazu verändert sich bei **dynamischen (globalen) Verfahren** die Anzahl der Seitenrahmen, die ein Prozeß zur Laufzeit belegt. Es existiert keine maximale Obergrenze.

4.4 Beispiel: UNIX-Speicherverwaltung

- erste Versionen: nur Swapping
- ab 3BSD: Paging + Swapping
- dynamische Strategie

Hauptspeicheraufteilung:



Die Verwaltung der Seitenrahmen wird mittels der sogenannten **Core Map** realisiert.

Die Core-Map enthält für **jeden** Seitenrahmen einen Eintrag à 16 Byte:

Zeiger auf Nachfolger	Zeiger auf Vorgänger	Disk block number	...	free Bit	USAGE- Bit	Lock- Bit
--------------------------	-------------------------	----------------------	-----	-------------	---------------	--------------

nur benutzt, falls der Seitenrahmen auf der free-list ist

Die freien Seitenrahmen werden mittels einer doppelt verketteten Liste, der „Free-List“, verwaltet, die in der Core-Map gespeichert ist:

free-Bit=0 $\Leftrightarrow$ Seitenrahmen ist auf der Free-List.

Strategie:

- ① Im Falle eines Seitenfehlers wird der erste freie Rahmen auf der Free-List belegt.
- ② Periodisches Auffüllen der Free-List: Seiten, die zu keinem Working-Set gehören, werden auf die Free-List gesetzt.

Der Pagedaemon wird alle 250 ms geweckt, um die Free-List aufzufüllen:

- Falls $| \text{free list} | < \text{lotsfree}$, beginnt der Pagedaemon solange Seiten auszulagern, bis mindestens lotsfree Seitenrahmen auf der Free-List sind.
- sonst: `sleep()`

Parameter $\text{lotsfree} \approx \frac{1}{4}$ Speichergröße

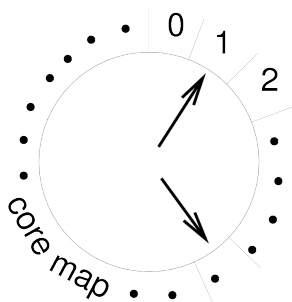
Welche Seiten gehören zu einem Working-Set bzw. zu keinem Working-Set?

Zeiger auf Nachfolger	Zeiger auf Vorgänger	Disk block number	...	free Bit	USAGE- Bit	Lock- Bit
--------------------------	-------------------------	----------------------	-----	-------------	---------------	--------------

nur benutzt, falls der Seitenrahmen auf der free-list ist

Ansatz: $\text{USAGE-Bit} = 0 \Leftrightarrow$ Seitenrahmen gehört zu keinem Working-Set.

Seitenersetzungsstrategie: "Two - handed clock algorithm"



Der 1. Zeiger löscht das USAGE-Bit.

Der 2. Zeiger prüft, ob das USAGE-Bit gelöscht ist.

- Falls das USAGE-Bit gelöscht $\implies$ Modifizierte Seiten werden auf den Hintergrundspeicher zurückgeschrieben und der Seitenrahmen an die Free-List gehängt.
- Pro Aufruf werden die Zeiger solange vorgerückt, bis genug Seitenrahmen auf der Free-List stehen.
maximal: weniger als 1 Umdrehung

Lokale oder Globale Strategie?

Bemerkung: Ein One-handed Clock Algorithm wurde schon in MULTICS benutzt.

Solomon/Russinovich: "On multiprocessor systems, Windows 2000 implements a variation of a local FIFO replacement policy. On uniprocessor systems, it implements something closer to a least recently used policy (LRU) (known as the clock algorithm, as implemented in most versions of UNIX).

Parameter für x86-Systeme: Seitengröße 4 KB

Seitenersetzungsstrategie:

- auf x86-Einprozessorsystemen: ähnlich Clock Algorithm: Falls Access-Bit im PTE (Page Table Entry) gelöscht ist, wird die Seite freigegeben, d.h. aus dem Working-Set entfernt. Andernfalls wird das Access-Bit gelöscht und die nächste Seite geprüft.
- auf Mehrprozessorsystemen und Alpha-Systemen: lokale FIFO-Seitenersetzungsstrategie

Seitenersetzungsstrategie

Definition 31:

Diejenigen Seiten des Prozesses, die im Hauptspeicher eingelagert sind, heißen **Working Set** des Prozesses („NT-Working-Set”).

Die Anzahl an Seiten, die ein Prozess mindestens im Hauptspeicher belegen darf, heißt **Minimum-Working-Set** des Prozesses, die maximale Anzahl heißt **Maximum-Working-Set**.

Hauptspeichergröße	
Small	≤ 19 MB
Medium	20-32 MB
Large	≥ 32 MB, falls Windows NT Workstation ≥ 64 MB, falls Windows NT Server

Haupt- speichergröße	Default Minimum-Working-Set Size (in Pages)	Default Maximum-Working-Set Size (in Pages)
Small	20	45
Medium	30	145
Large	50	345

Windows benutzt ein hybrides Verfahren, daß eine lokale und eine globale Strategie kombiniert:

- ① Unter „normalen Umständen“ erhält jeder Prozeß den Minimum-Working-Set und muß damit auskommen.
- ② Für einen Prozeß mit hoher Seitenfehlerrate wird der **aktuelle Working-Set** vergrößert, falls genügend freier Speicher vorhanden ist (Aufgabe des **Memory Managers**).

Variablen für das Erweitern des NT-Working-Sets

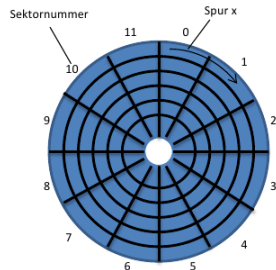
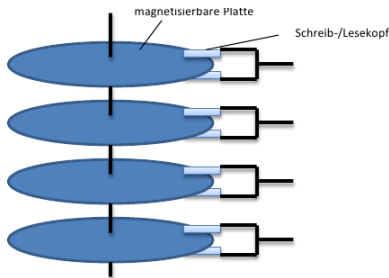
Variable	Wert	Bedeutung
MmWsExpandThreshold	90	Anzahl Seiten, die verfügbar sein müssen, damit der Working-Set über den Maximalwert erweitert werden darf.
MmWorkingSetSizeIncrement	6	Anzahl an Seiten, um die der Working-Set vergrößert wird, falls genügend Speicher vorhanden ist und der Working-Set unter dem Maximalwert liegt.
MmWorkingSetSizeExpansion	20	Anzahl an Seiten, um die der Working-Set vergrößert wird, falls genügend Speicher vorhanden ist und der Working-Set seinen Maximalwert erreicht hat.

Windows benutzt ein hybrides Verfahren, daß eine lokale und eine globale Strategie kombiniert:

- ① Unter „normalen Umständen“ erhält jeder Prozeß den Minimum-Working-Set und muß damit auskommen.
- ② Für einen Prozeß mit hoher Seitenfehlerrate wird der **aktuelle Working-Set** vergrößert, falls genügend freier Speicher vorhanden ist (Aufgabe des **Memory Managers**).
- ③ **Automatic Working Set Trimming**

Mit **Automatic Working Set Trimming** wird ein Verfahren bezeichnet, das versucht stets freie Seiten im System zu haben. Hierbei wird für jeden Prozeß geprüft, ob die Größe seines Working-Sets den Minimum-Working-Set-Wert überschreitet. Falls ja, werden dem Prozeß Seitenrahmen entzogen und somit sein Working-Set verkleinert. Die Zeit zwischen zwei Trimmzyklen beträgt 6s.

Physikalischer Aufbau Festplatte



Quelle: Mario Schölzel

Das mechanische Positionieren der Schreib-/Leseköpfe ist deutlich zeitaufwendiger als der eigentliche Lese- bzw. Schreibvorgang.

Demand-Paging with „Clustering“: Im Falle eines Seitenfehlers wird nicht nur die angeforderte Seite eingelagert, sondern auch eine gewisse Anzahl von Seiten aus der „Umgebung“.

Hauptspeichergröße	Clustergröße für Datenseiten		Clustergröße für Programmseiten	
	NT	2000	NT	2000
≤ 19 MB	0	2	1	3
> 19 MB	1-3	4	2-7	8

Strategie: Clock-Algorithm mit Optimierungen

1. Optimierung: Es soll möglichst zusammenhängender Speicher belegt werden, um effizientes Kopieren zwischen Haupt- und Hintergrundspeicher (Platte) zu ermöglichen.

Buddy-Liste `free_area`: Für $0 \leq i \leq 9$ enthält die `free_area` Listen mit 2^i zusammenhängenden freien Seiten

2. Optimierung: (stammt aus SunOS)

Ziel: den **internen** Verschnitt für **oft benutzte Datenstrukturen** minimieren

Beispiele: Einträge in der Prozesstabelle (Prozessleitblock), Eintrag in der Deskriptortabelle, ...

Slab Allocator: hält für häufig benutzte Datentypen passenden Speicherplatz vor

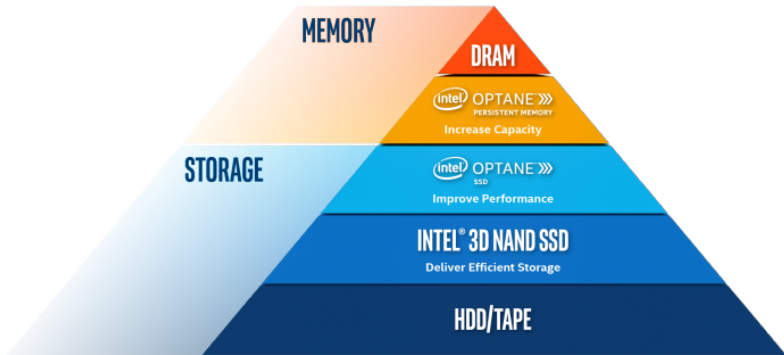
Ablauf:

- ① Der Slab Allocator holt sich freie Seitenrahmen von der Buddy-Liste.
- ② Der Slab Allocator schneidet einen Seitenrahmen in „Scheibchen“ passender Größe und verwaltet diese in Listen.
- ③ Speicherplatz, der frei gegeben wird, wird weiterhin vom Slab Allocator verwaltet und wiederverwendet.

(siehe auch `/proc/slabinfo`)



- Grundproblem: externe und interne Fragmentierung
- virtuelle Adressierung und Paging vermeiden externe Fragmentierung und ermöglichen es, Programme ablaufen zu lassen, die größer als der verfügbare Hauptspeicher sind.
- In der Theorie geeignet: LRU
- Approximativ umgesetzt im Working-Set-Verfahren
- Implementierung des Working-Set-Verfahrens mittels USAGE-Bit (UNIX, Two-handed Clock Algorithm) bzw. Access-Bit (Windows NT)
- Buddy-Verfahren: ursprünglich im Kontext von Swapping vorgeschlagen, um externe Fragmentierung zu reduzieren, wird eingesetzt
 - Linux: `free_area` Listen
 - Doug Lea's `dlmalloc()` für die Verwaltung des virtuellen Adressraums (siehe Übung)



Quelle: Intel

siehe auch [Intel Optane Persistent Memory aims to fill the gap DRAM can't, ComputerWeekly](#),

10.11.2020 ANNO 2022 von Intel abgekündigt.